

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/5_encoded.xlsx")

df_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/6_mice_context.xlsx")
df_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/7_mice_no_context.xlsx")

df_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/8_knn_context.xlsx")
df_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/9_knn_no_context.xlsx")

df_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/10_forest_context.xlsx")
df_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/11_forest_no_context.xlsx")
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Get the data types of each column
    data_types = df.dtypes

    # Create a DataFrame for the missing data summary including data types
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values,
        'Data Type': data_types.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage', 'Data Type']]
    group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage', 'Data Type']]
    group_4 = missing_data_summary[(missing_data_summary['Missing Percentage'] <= 20)][['Column', 'Missing Percentage', 'Data Type']]
```

```

group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage', 'Data Type']]
group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

# Sort each group by missing percentage in descending order (most missing to least missing)
group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

# Rename columns for clarity
group_1.columns = ['Column', 'Missing %', 'Data Type']
group_2.columns = ['Column', 'Missing %', 'Data Type']
group_3.columns = ['Column', 'Missing %', 'Data Type']
group_4.columns = ['Column', 'Missing %', 'Data Type']

# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))

```

```
missing_data_report(df_encoded)
```

```

Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

```

```

Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %  Data Type
      feels_like  53.061224   float64
      air_humidity  53.061224   float64
      wind_deg  53.061224   float64
      clouds_sky  53.061224   float64
      weather_main_encoded  52.999382   float64
      weather_description_encoded  52.999382   float64
      financial_condition_encoded  45.516388   float64

```

```

Group 3: Columns with >20% to ≤40% missing data
      Column  Missing %  Data Type
workplace_encoded_semi-urban  37.909709   float64
workplace_encoded_urban  37.909709   float64
workplace_encoded_rural  37.909709   float64
      air_pressure  30.426716   float64
      wind_speed  30.179344   float64
      temp_min  29.870130   float64
      temp_max  29.560915   float64
      marital_status_encoded  23.933210   float64

```

```

Group 4: Columns with ≤20% missing data
      Column  Missing %  Data Type
profession_group_encoded  18.923933   float64
      reason_encoded  16.821274   float64
      age  12.739641   float64

```

time_encoded	12.059369	float64
age_group_encoded	7.421150	float64
latitude	6.493506	float64
longitude	6.493506	float64
method_encoded	2.906617	float64
suicide_month	2.411874	float64
suicide_day	2.411874	float64
suicide_weekday_encoded	2.411874	float64
suicide_week	2.411874	float64
suicide_year	2.411874	float64
religion_encoded_buddhist	2.411874	float64
religion_encoded_muslim	2.411874	float64
religion_encoded_indigenous	2.411874	float64
religion_encoded_hindu	2.411874	float64
religion_encoded_christian	2.411874	float64
hometown_encoded	1.669759	float64
gender_encoded_third gender	0.432900	float64
gender_encoded_female	0.432900	float64
gender_encoded_male	0.432900	float64

```
def create_datasets(df_encoded):
    # Columns with >40% to <=70% missing (Group 2)
    group2_columns = [
        'feels_like', 'air_humidity', 'wind_deg', 'clouds_sky',
        'weather_main_encoded', 'weather_description_encoded', 'financial_condition_encoded'
    ]

    # Columns with >20% to <=40% missing (Group 3)
    group3_columns = [
        'workplace_encoded_semi-urban', 'workplace_encoded_urban', 'workplace_encoded_rural',
        'air_pressure', 'wind_speed', 'temp_min', 'temp_max', 'marital_status_encoded'
    ]

    # Columns with <=20% missing (Group 4)
    group4_columns = [
        'profession_group_encoded', 'reason_encoded', 'age', 'time_encoded',
        'age_group_encoded', 'latitude', 'longitude', 'method_encoded',
        'suicide_month', 'suicide_day', 'suicide_weekday_encoded', 'suicide_week',
        'suicide_year', 'religion_encoded_buddhist', 'religion_encoded_muslim',
        'religion_encoded_indigenous', 'religion_encoded_hindu', 'religion_encoded_christian',
        'hometown_encoded', 'gender_encoded_third gender', 'gender_encoded_female',
        'gender_encoded_male'
    ]

    # Dataset 1: Columns with <=20% missing
    df1_columns = group4_columns
    df1 = df_encoded[df1_columns].copy().dropna()

    # Dataset 2: Columns with <=40% missing (Group 3 + Group 4)
    df2_columns = group3_columns + group4_columns
    df2 = df_encoded[df2_columns].copy().dropna()

    # Dataset 3: All columns (Group 2 + Group 3 + Group 4)
    df3_columns = group2_columns + group3_columns + group4_columns
    df3 = df_encoded[df3_columns].copy().dropna()

    return df1, df2, df3
```

```
df_encoded_20, df_encoded_40, df_encoded_70 = create_datasets(df_encoded)
```

```
display(df_encoded_20.shape)
display(df_encoded_40.shape)
display(df_encoded_70.shape)
```

```
(853, 22)
(403, 30)
(57, 37)
```

```
df_mice_context = df_mice_context.dropna()
df_mice_no_context = df_mice_no_context.dropna()
df_knn_context = df_knn_context.dropna()
df_knn_no_context = df_knn_no_context.dropna()
df_rf_context = df_rf_context.dropna()
df_rf_no_context = df_rf_no_context.dropna()
```

```
display(df_mice_context.shape)
display(df_mice_no_context.shape)
display(df_knn_context.shape)
display(df_knn_no_context.shape)
display(df_rf_context.shape)
display(df_rf_no_context.shape)
```

```
(1613, 37)
(1613, 37)
(1617, 37)
(1617, 37)
(1617, 29)
(1617, 29)
```

✓ Leaving the df_encoded_70 dataframe since very low number of rows

```
dataframes = [df_encoded_20, df_encoded_40, df_mice_context, df_mice_no_context, df_knn_context, df_knn_no_context, df_rf_context, df_rf_no_context]
df_names = ["df_encoded_20", "df_encoded_40", "df_mice_context", "df_mice_no_context", "df_knn_context", "df_knn_no_context", "df_rf_context", "df_rf_no_context"]
```

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import SpectralClustering
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from tqdm import tqdm
import numpy as np
```

```
def preprocess_data(X, scale=True, pca_components=None):
    if scale:
        X = StandardScaler().fit_transform(X)
    if pca_components is not None and pca_components < X.shape[1]:
        X = PCA(n_components=pca_components, random_state=42).fit_transform(X)
    return X
```

```
def tune_spectral_fast(X, param_grid, sample_size=None):
```

```

"""Fast hyperparameter tuning with optional subsampling"""
if sample_size is not None and X.shape[0] > sample_size:
    rng = np.random.RandomState(42)
    idx = rng.choice(X.shape[0], size=sample_size, replace=False)
    X_tune = X[idx]
else:
    X_tune = X

best_score = -np.inf
best_params = None
best_labels = None
results = []

for n_clusters in tqdm(param_grid["n_clusters"], desc="Tuning clusters"):
    for affinity in param_grid["affinity"]:
        for assign_labels in param_grid["assign_labels"]:
            try:
                spc = SpectralClustering(
                    n_clusters=n_clusters,
                    affinity=affinity,
                    assign_labels=assign_labels,
                    random_state=42
                )
                labels = spc.fit_predict(X_tune)
                sil = silhouette_score(X_tune, labels)
                ch = calinski_harabasz_score(X_tune, labels)
                db = davies_bouldin_score(X_tune, labels)
                combined = sil + np.log(ch + 1) - db
                results.append({"n_clusters": n_clusters, "affinity": affinity,
                               "assign_labels": assign_labels, "sil": sil,
                               "ch": ch, "db": db, "score": combined})
                if combined > best_score:
                    best_score = combined
                    best_params = {"n_clusters": n_clusters, "affinity": affinity, "assign_labels": assign_labels}
                    best_labels = labels
            except Exception:
                continue
    return best_params, best_labels, results

def spectral_pipeline(df, scale=True, pca_components=20, tuning_sample=None):
    X = preprocess_data(df.values, scale=scale, pca_components=pca_components)

    param_grid = {
        "n_clusters": list(range(2, min(6, len(df)//2))),
        "affinity": ["nearest_neighbors"], # sparse affinity
        "assign_labels": ["kmeans"],
    }

    best_params, best_labels, all_results = tune_spectral_fast(X, param_grid, sample_size=tuning_sample)

    # Assign labels to existing df
    df["spc_cluster"] = best_labels

    return best_params, all_results

```

```

from tqdm import tqdm

best_params_results = {}
all_eval_results = {}

# Loop with progress bar
for df, name in tqdm(list(zip(dataframes, df_names)), desc="Processing DataFrames"):
    print(f"\n=== Processing {name} ===")

    best_params, all_results = spectral_pipeline(
        df,
        scale=True,          # False if already scaled
        pca_components=None  # or e.g., 10
    )

    # Save results
    best_params_results[name] = best_params
    all_eval_results[name] = all_results

print("\n✅ Finished all datasets!")
print("\nBest params per dataset:")
for name, params in best_params_results.items():
    print(name, ":", params)

# Now each original df already has 'spc_cluster' column added

```

```

Processing DataFrames:  0%|          | 0/8 [00:00<?, ?it/s]
=== Processing df_encoded_20 ===

Tuning clusters:  0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 25%|██        | 1/4 [00:00<00:00, 5.44it/s]
Tuning clusters: 50%|████      | 2/4 [00:00<00:00, 5.86it/s]
Tuning clusters: 75%|██████    | 3/4 [00:00<00:00, 5.19it/s]
Tuning clusters: 100%|████████ | 4/4 [00:00<00:00, 5.05it/s]
Processing DataFrames: 12%|██    | 1/8 [00:00<00:05, 1.25it/s]
=== Processing df_encoded_40 ===

Tuning clusters:  0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 50%|████      | 2/4 [00:00<00:00, 12.46it/s]
Tuning clusters: 100%|████████ | 4/4 [00:00<00:00, 11.48it/s]
Processing DataFrames: 25%|███   | 2/8 [00:01<00:03, 1.86it/s]
=== Processing df_mice_context ===

Tuning clusters:  0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 25%|██        | 1/4 [00:00<00:01, 2.21it/s]
Tuning clusters: 50%|████      | 2/4 [00:00<00:00, 2.81it/s]
Tuning clusters: 75%|██████    | 3/4 [00:01<00:00, 2.52it/s]
Tuning clusters: 100%|████████ | 4/4 [00:01<00:00, 2.59it/s]
Processing DataFrames: 38%|█████  | 3/8 [00:02<00:05, 1.00s/it]
=== Processing df_mice_no_context ===

Tuning clusters:  0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 25%|██        | 1/4 [00:00<00:00, 3.23it/s]
Tuning clusters: 50%|████      | 2/4 [00:00<00:00, 3.30it/s]
Tuning clusters: 75%|██████    | 3/4 [00:00<00:00, 3.30it/s]
Tuning clusters: 100%|████████ | 4/4 [00:01<00:00, 3.23it/s]
Processing DataFrames: 50%|█████  | 4/8 [00:03<00:04, 1.10s/it]
=== Processing df_knn_context ===

```

```

Tuning clusters: 0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 25%|██       | 1/4 [00:00<00:00, 3.97it/s]
Tuning clusters: 50%|██████   | 2/4 [00:00<00:00, 3.46it/s]
Tuning clusters: 75%|██████████| 3/4 [00:00<00:00, 3.23it/s]
Tuning clusters: 100%|██████████| 4/4 [00:01<00:00, 3.25it/s]
Processing DataFrames: 62%|██████ | 5/8 [00:05<00:03, 1.15s/it]
=== Processing df_knn_no_context ===

```

```

Tuning clusters: 0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 25%|██       | 1/4 [00:00<00:00, 3.38it/s]
Tuning clusters: 50%|██████   | 2/4 [00:00<00:00, 3.71it/s]
Tuning clusters: 75%|██████████| 3/4 [00:00<00:00, 3.65it/s]
Tuning clusters: 100%|██████████| 4/4 [00:01<00:00, 3.41it/s]
Processing DataFrames: 75%|██████ | 6/8 [00:06<00:02, 1.16s/it]
=== Processing df_rf_context ===

```

```

Tuning clusters: 0%|          | 0/4 [00:00<?, ?it/s]
Tuning clusters: 25%|██       | 1/4 [00:00<00:00, 3.82it/s]
Tuning clusters: 50%|██████   | 2/4 [00:00<00:00, 3.39it/s]
Tuning clusters: 75%|██████████| 3/4 [00:00<00:00, 3.00it/s]
Tuning clusters: 100%|██████████| 4/4 [00:01<00:00, 3.01it/s]
Processing DataFrames: 88%|██████ | 7/8 [00:07<00:01, 1.22s/it]
=== Processing df_rf_no_context ===

```

```

from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from scipy.spatial.distance import cdist
import numpy as np

```

```

def evaluate_clusters(
    df,
    label_col,
    ignore_label=None,      # e.g., -1 for DBSCAN, None for KMeans/Agglo/KNN
    sil_metric="euclidean", # match your clustering distance
    sample_size=None,
    random_state=42,
    return_dict=False
):
    """
    General-purpose clustering evaluation function.
    Works for Agglomerative, DBSCAN, KMeans, Spectral, GMM, etc.
    """

    rng = np.random.RandomState(random_state)

    X = df.drop(columns=[label_col]).values
    labels = df[label_col].values

    # Optionally ignore a label (e.g., DBSCAN noise = -1)
    if ignore_label is not None:
        mask = labels != ignore_label
        X, labels = X[mask], labels[mask]

    # Optional subsample for speed (applies to all metrics)
    if sample_size is not None and X.shape[0] > sample_size:
        idx = rng.choice(X.shape[0], size=sample_size, replace=False)
        X, labels = X[idx], labels[idx]

```

```

unique_clusters = np.unique(labels)
n_clusters = len(unique_clusters)

if n_clusters <= 1:
    print("⚠ Only one cluster found. Metrics cannot be computed robustly.")
    return {} if return_dict else None

results = {}

# Core indices
try:
    results["Silhouette Score"] = {
        "score": silhouette_score(X, labels, metric=sil_metric),
        "range": "[-1, 1]",
        "better": "higher"
    }
except Exception:
    results["Silhouette Score"] = {"score": np.nan, "range": "[-1, 1]", "better": "higher"}

try:
    results["Calinski-Harabasz Index"] = {
        "score": calinski_harabasz_score(X, labels),
        "range": "[0, ∞)",
        "better": "higher"
    }
except Exception:
    results["Calinski-Harabasz Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

try:
    results["Davies-Bouldin Index"] = {
        "score": davies_bouldin_score(X, labels),
        "range": "[0, ∞)",
        "better": "lower"
    }
except Exception:
    results["Davies-Bouldin Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

# Dunn Index (guard against singletons / zero intra-distances)
def dunn_index(X, labels):
    eps = 1e-12
    uniq = np.unique(labels)
    # Intra-cluster diameters
    intra = []
    for k in uniq:
        pts = X[labels == k]
        if pts.shape[0] >= 2:
            d = cdist(pts, pts)
            intra.append(np.max(d))
        else:
            intra.append(0.0) # singleton cluster
    denom = max(max(intra), eps)

    # Inter-cluster minimum distance
    inter_min = np.inf
    for i in range(len(uniq)):
        for j in range(i + 1, len(uniq)):

```



```

        Xi = X[labels == uniq[i]]
        Xj = X[labels == uniq[j]]
        d = cdist(Xi, Xj)
        inter_min = min(inter_min, np.min(d))
    if not np.isfinite(inter_min):
        return np.nan
    return inter_min / denom

# Xie-Beni Index (guard against identical centroids)
def xie_beni_index(X, labels):
    eps = 1e-12
    uniq = np.unique(labels)
    centroids = np.array([X[labels == k].mean(axis=0) for k in uniq])
    # Compactness
    compact = 0.0
    for i, k in enumerate(uniq):
        pts = X[labels == k]
        diff = pts - centroids[i]
        compact += np.sum(diff * diff)
    # Min centroid separation
    min_csep = np.inf
    for i in range(len(centroids)):
        for j in range(i + 1, len(centroids)):
            d = np.sum((centroids[i] - centroids[j]) ** 2)
            if d < min_csep:
                min_csep = d
    if not np.isfinite(min_csep) or min_csep <= eps:
        return np.nan
    return compact / (X.shape[0] * min_csep)

try:
    results["Dunn Index"] = {
        "score": dunn_index(X, labels),
        "range": "[0, ∞)",
        "better": "higher"
    }
except Exception:
    results["Dunn Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

try:
    results["Xie-Beni Index"] = {
        "score": xie_beni_index(X, labels),
        "range": "[0, ∞)",
        "better": "lower"
    }
except Exception:
    results["Xie-Beni Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

# Pretty print
print("\n📊 Clustering Evaluation Results:")
for metric, info in results.items():
    score = info['score']
    score_str = f"{score:.4f}" if isinstance(score, (int, float)) and np.isfinite(score) else "NaN"
    print(f"{metric}: {score_str} ----- Range: {info['range']} ----- {info['better'].capitalize()} is better")

```

```
return results if return_dict else None
```

```
for df, name in zip(dataframes, df_names):
    print("\n-----")
    print(f"Evaluation metrics for {name}: total clusters: {df['spc_cluster'].nunique()}")
    print("=====")

    results = evaluate_clusters(
        df,
        label_col='spc_cluster', # use the Spectral Clustering column
        ignore_label=None,      # no noise label for Spectral Clustering
        sil_metric="euclidean",
        sample_size=None,       # full dataset (no subsampling)
        return_dict=True
    )
```

```
-----
Evaluation metrics for df_encoded_20: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.0121 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 1.3721 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 15.3688 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0256 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 65.8483 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_encoded_40: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.9036 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 7801.5723 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.1165 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0053 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0109 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_mice_context: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.6708 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 5302.2638 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.4260 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0018 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0759 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_mice_no_context: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.7878 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 12376.4397 ----- Range: [0, ∞) ----- Higher is better
```

```

Davies-Bouldin Index: 0.2671 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0212 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0322 ----- Range: [0, ∞) ----- Lower is better

-----
Evaluation metrics for df_knn_context: total clusters: 2
=====

📊 Clustering Evaluation Results:
Silhouette Score: 0.7949 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 9833.8152 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.2492 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0257 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0317 ----- Range: [0, ∞) ----- Lower is better

```

```

# from google.colab import files
# import pandas as pd

# # Assuming dataframes and df_names are already defined and populated
# # from previous cells.

# output_dir = "/content/spc_exported_dfs"
# os.makedirs(output_dir, exist_ok=True)

# exported_files = []

# for df, name in zip(dataframes, df_names):
#     # Construct the output filename
#     output_filename = f"spc_{name}.xlsx"
#     output_filepath = os.path.join(output_dir, output_filename)

#     # Export the dataframe to an Excel file
#     try:
#         df.to_excel(output_filepath, index=False)
#         print(f"Exported {name} to {output_filepath}")
#         exported_files.append(output_filepath)
#     except Exception as e:
#         print(f"Error exporting {name}: {e}")

# # Provide a way to download the files
# if exported_files:
#     print("\nYour files are ready for download:")
#     for f in exported_files:
#         print(f)
#     # You can manually download from the file browser in Colab,
#     # or use the following code to download them programmatically:
#     # for f in exported_files:
#     #     files.download(f)
# else:
#     print("\nNo files were exported.")

```

```

Exported df_encoded_20 to /content/spc_exported_dfs/spc_df_encoded_20.xlsx
Exported df_encoded_40 to /content/spc_exported_dfs/spc_df_encoded_40.xlsx
Exported df_mice_context to /content/spc_exported_dfs/spc_df_mice_context.xlsx
Exported df_mice_no_context to /content/spc_exported_dfs/spc_df_mice_no_context.xlsx
Exported df_knn_context to /content/spc_exported_dfs/spc_df_knn_context.xlsx

```

```
Exported df_knn_no_context to /content/spc_exported_dfs/spc_df_knn_no_context.xlsx  
Exported df_rf_context to /content/spc_exported_dfs/spc_df_rf_context.xlsx  
Exported df_rf_no_context to /content/spc_exported_dfs/spc_df_rf_no_context.xlsx
```

Your files are ready for download:

```
/content/spc_exported_dfs/spc_df_encoded_20.xlsx  
/content/spc_exported_dfs/spc_df_encoded_40.xlsx  
/content/spc_exported_dfs/spc_df_mice_context.xlsx  
/content/spc_exported_dfs/spc_df_mice_no_context.xlsx  
/content/spc_exported_dfs/spc_df_knn_context.xlsx  
/content/spc_exported_dfs/spc_df_knn_no_context.xlsx  
/content/spc_exported_dfs/spc_df_rf_context.xlsx  
/content/spc_exported_dfs/spc_df_rf_no_context.xlsx
```